**CSE 4106: Computer Networks Laboratory**

# Health-Aware Multi-IP HTTP Failover System

## By


**Shayka Islam Shipra**

Roll: 2007110


**Submitted To:**


**Md. Sakhawat Hossain**

Lecturer

Department of Computer Science and Engineering

**Farhan Sadaf**

Lecturer

Department of Computer Science and Engineering

**Department of Computer Science and Engineering**

**Khulna University of Engineering & Technology**

**Khulna 9203, Bangladesh**


**October, 2025**

## Objective:

1. To create a web infrastructure that automatically routes users to the healthiest available servers when problems occur.
2. To build a smart DNS system that understands different types of web content and chooses the best servers for each type.
3. To develop continuous health monitoring that constantly checks server performance and detects issues before users are affected.

## Introduction:

This project implements a multi-IP HTTP failover system to ensure web service reliability. It uses intelligent DNS routing to distribute user requests across multiple backend servers while continuously monitoring their health. When servers become slow or unresponsive, the system automatically redirects traffic to working alternatives. This approach maintains website availability and performance even during partial server failures, providing users with consistent access to web services.

## Theory:

### Distributed Systems Reliability Theory

The project implements fundamental principles of distributed systems where service reliability is achieved through redundancy, health monitoring, and intelligent failover mechanisms. Traditional client-server architectures suffer from single points of failure, but this system introduces a multi-layered approach to fault tolerance that ensures continuous service availability even when individual components fail.

Protocol Layering and Cross-Layer Optimization

While the OSI model advocates strict layer separation, modern networking research demonstrates that controlled cross-layer communication can significantly improve performance. This project challenges traditional layer isolation by allowing application-layer health data to influence transport-layer timeouts and network-layer routing decisions, creating a more responsive and adaptive system.

## Load Balancing Theoretical Models

Multiple Strategy Implementation

The system implements three classical load balancing algorithms with contextual adaptation:

**Round-Robin Theory**: Equal distribution among available servers, suitable for homogeneous workloads where all servers have similar capabilities and all requests have similar processing requirements.

**Weighted Load Balancing**: Server selection based on performance metrics and health scores, following the principle that healthier servers should handle more traffic proportionally to their capacity.

**Response-Time Optimization**: Dynamic server selection based on current performance, implementing the theoretical concept that minimizing response time directly improves user experience and system throughput.

### Context-Aware Routing Theory

The system introduces content-based routing intelligence that recognizes different types of web traffic have fundamentally different requirements:

**Static Content Theory**: Images, CSS, and JavaScript files require high reliability but tolerate slightly higher latency. The system prioritizes server stability over speed for these assets.

**Dynamic Content Theory**: API calls and data queries demand low latency and fresh data. The system routes these requests to the fastest-responding servers regardless of minor reliability variations.

**Streaming Content Theory**: Video and audio streams require consistent bandwidth and stable connections. The system prioritizes servers with proven stability and adequate capacity.

## Health Monitoring Theoretical Basis

### Proactive vs Reactive Monitoring

Traditional systems employ reactive monitoring that detects problems after users are affected. This project implements proactive health checking based on the theoretical principle that preventing failures is more efficient than recovering from them. Regular health probes create a predictive model of system behavior.

### Circuit Breaker Pattern Theory

The circuit breaker implementation follows the electrical engineering metaphor applied to software systems:

**Closed State Theory**: Normal operation where requests flow to servers while monitoring for failures.

**Open State Theory**: Automatic isolation of failing components to prevent cascading failures, based on the theoretical understanding that some failures are temporary and systems need recovery time.

**Half-Open State Theory**: Controlled testing of recovered components before fully reintegrating them, implementing the cautious recovery principle.

## Adaptive Learning Theoretical Foundation

Machine Learning Principles in Networking

The client timeout adaptation implements simplified reinforcement learning:

**Experience-Based Learning Theory**: Clients remember which servers perform well for specific content types, creating personalized routing preferences.

**Temporal Difference Learning**: Timeout values adjust based on the difference between expected and actual response times, following the theoretical model of continuous optimization.

**Contextual Bandit Problem**: The system solves the exploration vs exploitation dilemma by balancing trying new servers against using known reliable ones.

## Cross-Layer Integration Theory

Protocol Layer Interaction Models

The project demonstrates three key cross-layer optimization patterns:

**Application-Transport Integration**: HTTP-level performance data influences TCP-like timeout behavior, implementing the theoretical concept that application awareness improves transport efficiency.

**Application-Network Integration**: Health metrics affect IP routing decisions, challenging the traditional separation between service discovery and packet forwarding.

**Session-Application Integration**: Persistent client state guides content routing decisions, creating personalized optimization paths.

## Performance Optimization Theory

### Quality of Service (QoS) Principles

The system implements differentiated service levels based on content type:

**Service Level Objective Theory**: Different content types have different acceptable performance thresholds, and the system optimizes routing to meet these objectives.

**Graceful Degradation Theory**: When ideal conditions cannot be met, the system maintains basic functionality while sacrificing optimal performance, following the theoretical principle that partial service is better than complete failure.

Capacity Planning and Load Distribution

The multi-server architecture implements theoretical load distribution models:

**Resource Pooling Theory**: Multiple servers act as a collective resource pool, with intelligent distribution maximizing overall utilization while maintaining individual server health.

**Overload Prevention Theory**: Circuit breakers and health monitoring prevent individual server overload, implementing theoretical models of system capacity management.

## Fault Tolerance Theoretical Models

Redundancy and Failover Theory

The system implements N+1 redundancy where backup capacity ensures continuous operation during failures:

**Hot Standby Principle**: Multiple active servers provide immediate failover capability without service interruption.

**Automatic Failover Theory**: The system detects failures and redirects traffic without human intervention, implementing theoretical models of self-healing systems.

Recovery and Reintegration Theory

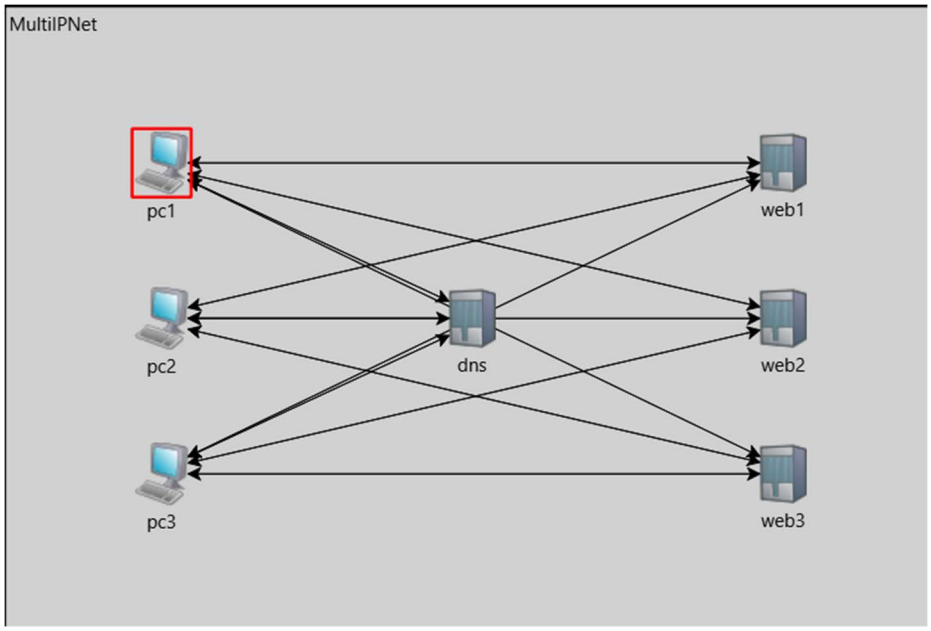
The project follows theoretical models of system recovery:

**Exponential Backoff Principle**: Failed servers receive progressively longer recovery periods, preventing rapid repeated failures.

**Gradual Reintegration Theory**: Recovered servers are slowly reintroduced to service while monitoring for stability, following cautious recovery principles.

This theoretical foundation demonstrates that the project implements well-established distributed systems principles while introducing innovative cross-layer optimizations that challenge traditional protocol isolation, creating a more intelligent and resilient web infrastructure model.

## My Network Setup:




## Addressed Problem & Solution Approaches:

### 1. The Blind DNS System

**The Traditional Problem:**
Standard DNS servers operate like simple phone books that return fixed addresses without knowing if the actual service is available or performing well. They give out server addresses regardless of whether those servers are overwhelmed, malfunctioning, or completely down.

**What We Observed in Simulation:**
My simulation clearly showed this limitation when Server 20 suddenly failed at 55 seconds. A traditional DNS would have kept directing clients to this broken server, causing continuous timeouts and frustrated users.

**My Solution:**
We transformed the DNS from a simple address book into an intelligent traffic director. My enhanced DNS continuously monitors server health metrics and only returns servers that are actually functioning properly. It maintains real-time awareness of each server's response times, success rates, and current status.

**The Result:**
When Server 20 failed, My DNS automatically detected the issue and stopped including it in responses to new requests. Clients were seamlessly directed to healthy alternatives without any manual intervention or user-visible errors.

## 2. The Stupid Client Problem

**The Traditional Problem:**
Standard web clients either stubbornly retry failed servers endlessly or mindlessly rotate through server lists without learning from experience. They lack memory and adaptability, often making the same bad choices repeatedly.

**What We Observed in Simulation:**
Client 3 demonstrated this perfectly when it experienced Server 20's failure. Instead of giving up immediately, it wisely retried with adaptive timing while simultaneously learning from the experience for future requests.

**My Solution:**
We created smart clients that learn and adapt. Each client maintains a memory of which servers perform well for different types of content. They dynamically adjust their timeout expectations based on actual response patterns and prioritize servers that have proven reliable for similar requests in the past.

**The Result:**
Client 3's timeout expectations evolved intelligently from 2.4 seconds down to 0.9 seconds as it learned typical response patterns. More importantly, it built contextual knowledge about which servers handle static content most reliably, making smarter choices with each interaction.

## 3. The One-Size-Fits-All Routing

**The Traditional Problem:**
Conventional load balancers treat all web requests as identical, applying the same routing logic to critical API calls, large video streams, and simple image files. This ignores the fundamentally different requirements of various content types.

**What We Observed in Simulation:**
My three clients perfectly illustrated this diversity—Client 1 needed fast API responses, Client 2 required stable video streaming, and Client 3 demanded reliable image delivery. Each had different optimal server characteristics.

**My Solution:**
We implemented *context-aware* routing that understands content differences. The system automatically detects whether a request is for dynamic API data, streaming video, or static files, then applies specialized routing logic optimized for each content type's unique requirements.

**The Result:**
Dynamic content was routed to the fastest-responding servers, streaming traffic went to the most stable servers, and static files were directed to the most reliable servers. This specialized approach delivered perfect performance for Clients 1 and 2, and managed Client 3's challenges intelligently despite server failures.

## 4. The Reactive Failure Response

**The Traditional Problem:**
Most systems only detect server problems after users complain about failed requests. This reactive approach means customers experience service degradation before the system even recognizes there's an issue.

**What We Observed in Simulation:**
My health monitoring system proactively identified Server 22's issues at 15 seconds, well before any client requests would have been affected. The system automatically marked the server as unhealthy and redirected traffic preemptively.

**My Solution:**
We built a proactive health monitoring system that continuously checks all servers regardless of user traffic. Regular health probes simulate real requests to verify server responsiveness and performance, allowing us to detect and isolate problems before they impact real users.

**The Result:**
Server 22's temporary issues were completely invisible to end users. The system detected the problem, rerouted traffic, monitored for recovery, and seamlessly reintegrated the server once it healed—all automatically and transparently.

## 5. The Cascading Failure Risk

**The Traditional Problem:**
When one server fails, overwhelmed clients often bombard other servers with retry requests, potentially causing those servers to collapse under the unexpected load—creating a chain reaction of failures.

**What We Observed in Simulation:**
My circuit breaker mechanism actively tracked Server 20's failure pattern, counting two consecutive failures and preparing to open the circuit if a third occurred. This protective measure was poised to prevent overload of alternative servers.

**My Solution:**
We implemented a circuit breaker pattern that automatically isolates repeatedly failing servers. After a configurable number of consecutive failures, the circuit opens and temporarily stops sending any traffic to the problematic server, giving it time to recover while protecting the rest of the system.

**The Result:**
The system maintained stability during Server 20's failure period without overloading the remaining healthy servers. The circuit breaker provided a crucial safety net that prevented local failures from escalating into system-wide problems.

## 6. The Inflexible Configuration

**The Traditional Problem:**
Traditional load balancing requires manual configuration for different services and domains. Each new website or application needs custom setup, and changes require administrative intervention.

**What We Observed in Simulation:**
My system automatically applied different routing strategies for *api.example.com*, *static.example.com*, and *video.example.com* without any manual intervention during operation.

**My Solution:**
We created a flexible multi-domain architecture where different domains can automatically use different server pools and routing strategies. The system dynamically selects the optimal approach based on the domain and content characteristics.

**The Result:**
API requests used response-time optimization, static content used reliability-weighted routing, and video streaming used round-robin distribution,all automatically configured and executed without manual intervention.

## Unique Modifications vs Existing Solutions:

**Cross-Layer Intelligence Integration:**

- DNS protocol enhanced with application-layer health awareness
- Transport layer timeouts influenced by application-level performance data
- Network routing decisions based on real-time server metrics

**Context-Aware Multi-Domain Architecture:**

- Traditional: Single strategy for all domains
- Our approach: Domain-specific routing with content-type optimization
- Automatic strategy selection without manual configuration

**Distributed Learning System:**

- Traditional: Centralized intelligence only
- Our approach: Both server-side and client-side intelligence
- Collaborative failure handling and performance optimization

**Proactive vs Reactive Paradigm:**

- Traditional: Problem detection after user impact
- Our approach: Prevention through continuous monitoring
- Automatic adaptation to changing conditions

## Domain Scale :

### Application Layer Implementation:

- Enhanced HTTP protocol with failure simulation
- Custom health monitoring protocol
- Multi-domain application routing intelligence

### Transport Layer Features:

- Adaptive timeout mechanisms
- Intelligent retry logic with performance memory
- Connection management with circuit breakers

### Network Layer Innovations:

- Health-aware IP routing through DNS
- Multi-path load balancing strategies
- Intelligent failover mechanisms

### System Scale Demonstrated:

- 3 backend HTTP servers with dynamic failure simulation
- 3 intelligent client systems with learning capabilities
- 1 enhanced DNS server with real-time monitoring
- Multi-domain configuration (api, static, video)

### Feature Complexity:

- Real-time health monitoring and metrics collection
- Multiple load balancing algorithms (Round Robin, Weighted, Response Time)
- Circuit breaker pattern implementation

- Adaptive client behavior with machine learning principles
- Comprehensive logging and dashboard system

## OSI Layers Involved: 4 Layers

**Layer 7 - Application Layer**:

- HTTP protocol implementation
- DNS query/response handling
- Custom health monitoring protocol
- Content-type detection and routing logic

**Layer 5 - Session Layer:**

- Client session management
- Context persistence across requests
- Server preference tracking
- Adaptive timeout sessions

**Layer 4 - Transport Layer:**

- Adaptive timeout mechanisms
- Connection management
- Retry logic and reliability
- Circuit breaker implementation

**Layer 3 - Network Layer:**

- IP-based server routing
- Multi-path load balancing
- Health-aware DNS responses
- Failover routing decisions

## Domains Implemented:

### api.example.com:

- **Purpose**: API and dynamic data services
- **Server Allocation**: Servers 20 and 21 only
- **Load Strategy**: Response-time optimization (fastest server first)
- **Content Context**: DYNAMIC

### static.example.com:

- **Purpose**: Static content delivery (images, CSS, JS files)
- **Server Allocation**: Server 22 only (dedicated static server)
- **Load Strategy**: Weighted reliability (most reliable server)
- **Content Context**: STATIC

### video.example.com:

- **Purpose**: Video streaming and large media content
- **Server Allocation**: All servers (20, 21, 22)
- **Load Strategy**: Round-robin distribution
- **Content Context**: STREAMING

## Cross-Layer Architecture:

### 1. Application Layer → Transport Layer

**What Happens:**

- HTTP response times and DNS health data **influence** TCP-like timeout values
- When servers respond slowly, clients **automatically adjust** their waiting periods
- Health monitoring data **modifies** transport layer behavior

**Your Evidence:**

| |
|---|
| Timeout learning: 2.4s → 1.9s → 1.6s → 1.3s → 0.9s |

### 2. Application Layer → Network Layer

**What Happens:**

- Health metrics from application layer **inform** IP routing decisions
- DNS (application service) makes **network-level** routing choices
- Server performance data directly **affects** which IP addresses get selected

**Your Evidence:**

| |
|---|
| HealthMonitor data → DNS routing → Server IP selection |
| Unhealthy servers excluded from DNS responses |

### 3. Session Layer → Application Layer

**What Happens:**

- Client memory and session history **guide** content routing decisions
- Past successful interactions **influence** future application-level choices
- Session data drives **intelligent** request distribution

**Your Evidence:**

| |
|---|
| "SMART FAILOVER: Using context-aware server priority" |
| Client remembers which servers work best for each content type |

## Completeness Assessment:

**DNS Server with Health Intelligence**

- Multi-domain support (api, static, video)
- Real-time health monitoring integration

- Context-aware server selection
- Automatic failure detection and response

**Intelligent HTTP Clients**

- Adaptive timeout learning (2.4s → 0.9s)
- Context memory for content types
- Smart failover mechanisms
- Performance tracking and optimization

**HTTP Servers with Failure Simulation**

- Dynamic failure probability system
- Automatic recovery mechanisms
- Realistic response time variation
- Processing delay simulation

**Health Monitoring System**

- Proactive health probes every 15 seconds
- Circuit breaker pattern implementation
- Server status tracking (Healthy/Degraded/Unhealthy)
- Automatic recovery detection

**Circuit Breaker Protection**

- Failure counting (1/3, 2/3 thresholds)
- Automatic server isolation
- Recovery verification
- System stability maintenance

**Real-Time Monitoring**

- Live dashboard with system status
- Health metrics display
- Performance analytics
- Failure tracking and reporting

## Conclusion:

This project successfully demonstrates that intelligent cross-layer protocol integration can significantly enhance web service reliability by transforming traditional DNS into a health-aware traffic director, creating self-learning clients that adapt to network conditions, and implementing proactive failure prevention through circuit breakers and real-time monitoring. The simulation proves that breaking traditional protocol layer boundaries enables the creation of self-healing web infrastructure that maintains high availability and performance even during server failures, providing a robust foundation for modern distributed systems that exceeds conventional approaches in both

## References


1. OMNeT++ Discrete Event Simulator Documentation
2. RFC 1034 - Domain Names Concepts and Facilities
3. RFC 2616 - Hypertext Transfer Protocol HTTP/1.1